

Java abstract classes and interfaces

What each is, what it can hold, how they differ, and how to choose between them (and against inheritance). Definitions and capability questions stay tight; the design and experience ones are how I would talk through them.

1. What is an abstract class?

An **abstract class** is a **partially complete class that cannot be instantiated on its own**; it is meant to be extended. It can mix **abstract methods** (declared, no body, subclass must implement) with **concrete methods and state**, so it gives subclasses a shared base plus a contract for the parts they must fill in.

2. What is an interface?

An **interface** is a **pure contract**: a set of method signatures a class promises to provide, with no instance state. A class **implements** it, and a class can implement many. Since Java 8 an interface can also carry default and static methods, so it is not strictly abstract-only anymore, but its core role is defining "what a type can do" without saying how.

3. What is an abstract method, and what is a default method?

An **abstract method** is declared without a body (`void save();`) and must be implemented by the first concrete subclass or implementer. A **default method** (interface, Java 8) is the opposite: a method **with a body** inside an interface, marked `default` , that implementers inherit for free and may override.

4. What is the difference between an abstract class and an interface?

	Abstract class	Interface
Purpose	Shared base for closely related types ("is-a")	Contract for a capability ("can-do")
Inheritance	A class extends one	A class implements many
State	Can hold instance fields	Only <code>public static final</code> constants, no instance state
Constructor	Yes	No
Methods	Abstract, concrete, static, final, private	Abstract, default, static, private (no <code>final</code>)
Access modifiers	Any (public, protected, private)	Members are implicitly public

The one-line version: an abstract class shares **implementation and state** among related classes, an interface defines a **capability** that unrelated classes can all provide.

5. When should you use an abstract class, and when an interface?

Use an **abstract class** when the implementers are **closely related and share real code or state**, and you want to hand them a common base (a `Shape` base for `Circle` and `Square`). Use an **interface** when you are

defining a **capability that unrelated types can offer** (Comparable , Serializable), or when a type needs to play **several roles at once**, since a class can implement many interfaces but extend only one class. In modern design the default is to **prefer interfaces** and only reach for an abstract class when there is genuine shared implementation.

6. What can an abstract class contain and do?

Nearly everything a normal class can, which is the point. Rapid-fire:

Can it...	Answer
Have constructors	Yes (run via <code>super()</code> when a subclass is built)
Be instantiated	No
Have concrete methods	Yes
Have static / final / private methods	Yes to all
Have instance variables and static variables	Yes to both
Exist with no abstract methods	Yes
Implement an interface	Yes (and may leave the methods abstract for subclasses)
Extend another abstract class	Yes
Extend a concrete class	Yes

7. Why would you declare a class abstract if it has no abstract methods?

To **stop it being instantiated directly** while still sharing its code. A common case is a base class that is complete but meaningless on its own (a generic `HttpServlet` -style base): marking it abstract forces callers to subclass it and use it through a concrete type, rather than creating a bare, half-configured instance.

8. Why can an abstract class have constructors even though it cannot be instantiated?

Because the constructor is **not for instantiating the abstract class directly; it runs as part of building a subclass**. When you create a subclass instance, its constructor calls `super()`, which runs the abstract class's constructor to **initialise the shared fields**. So the constructor exists to set up the inherited state, not to make a standalone object.

9. What are the advantages and limitations of abstract classes?

Advantages: they share **code and state** across related subclasses (less duplication), can evolve by adding concrete methods without breaking subclasses, and can control construction and access (protected members, constructors). **Limitations:** a class can extend **only one**, so they do not compose; deep abstract hierarchies get **rigid and hard to change**; and they force an "is-a" relationship that is often the wrong model.

10. What members can an interface contain, and why are its variables `public static final` ?

An interface can contain **constants**, **abstract methods**, **default methods** (Java 8), **static methods** (Java 8), and **private methods** (Java 9). It **cannot** have constructors or instance state.

Member	Allowed
<code>public static final</code> constants	Yes (fields are implicitly this)
Abstract methods	Yes
Default / static methods	Yes (Java 8)
Private methods	Yes (Java 9)
Constructors	No
Instance fields	No

Interface variables are **implicitly `public static final`** (so `int MAX = 10;` is a public constant) because an interface has **no instance to hold mutable state**; it is a contract, not an object, so any field can only be a shared constant. This is why "interface fields are always constants" is a rule to remember.

11. Why were default methods introduced in Java 8, and private methods in Java 9?

Default methods were introduced to let an interface **gain new methods without breaking every existing implementer**. Before Java 8, adding a method to an interface broke all implementing classes; the Java 8 team needed to add methods like `stream()` to `Collection`, so default let them ship a body that existing classes inherit for free. **Private methods** (Java 9) followed because once interfaces had default methods, those defaults needed **shared helper logic** that should not be exposed in the public contract, so private interface methods let defaults reuse code without leaking it.

12. Interface inheritance: what can extend or implement what?

- An interface **can extend another interface**, and **multiple interfaces** (`interface C extends A, B`).
- An interface **cannot extend a class** (it has no implementation to inherit).
- A class **can implement multiple interfaces**, which is how Java gets multiple inheritance of type without the diamond problems of multiple class inheritance.

13. What are the advantages and limitations of interfaces?

Advantages: **multiple inheritance of type**, **loose coupling** (code depends on the contract, not a concrete class), easy to mock in tests, and clean support for polymorphism. **Limitations:** **no instance state**, so shared data has to live elsewhere; before Java 8 they could not carry any implementation; and it is easy to create bloated "god interfaces" that force implementers to define methods they do not need.

14. Abstract class v/s interface, across the practical axes.

Axis	Abstract class	Interface
Speed	Effectively the same; not a real deciding factor	Effectively the same
Memory	Can hold state, so instances carry fields	No instance state of its own
Flexibility	Lower (single inheritance)	Higher (implement many)
Multiple inheritance	No	Yes (of type)
Ease of evolution	Add concrete methods freely	Add default methods (since Java 8)
Code reuse	Strong (shared implementation + state)	Limited (defaults only, no state)
State	Yes	No
Constructors	Yes	No
Multiple implementations	Via subclasses (one parent each)	A class can satisfy many interfaces

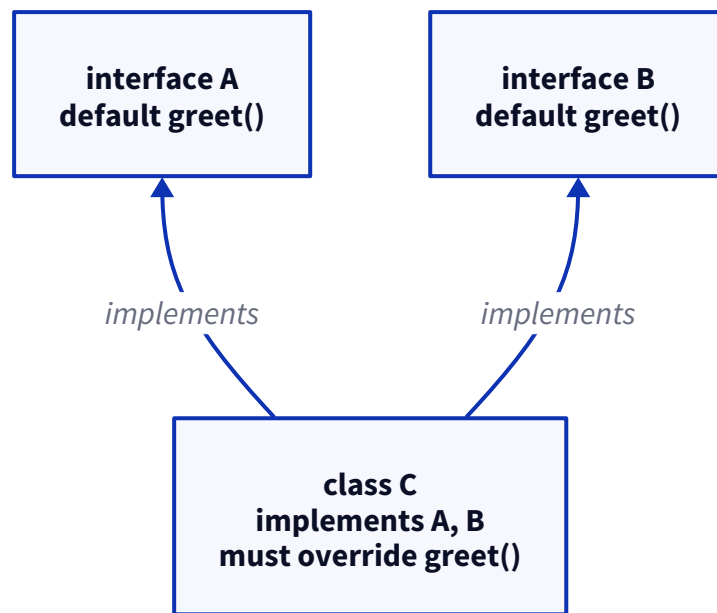
Guidance that follows from this: for a **library or framework API**, prefer **interfaces** so callers are not locked into your hierarchy (with an optional abstract "skeleton" base for convenience, the way the JDK pairs `List` with `AbstractList`). **Spring** leans heavily on interfaces (for proxying and dependency injection), and **microservices** favour interfaces for the same loose-coupling and testability reasons. Reach for an abstract class only when there is real shared implementation to inherit.

15. Default methods: what problem they solved, how they differ from abstract methods, and when to avoid them.

The problem they solved is **interface evolution** (see Q11): adding a method without breaking implementers. They differ from **abstract methods** in that a default method **has a body and is optional to override**, while an abstract method has no body and **must** be implemented. Default methods **can be overridden**, and they **can call private interface methods** for shared logic. When to **avoid** them: do not use defaults to carry real **business logic**; they are meant for backward-compatible convenience and sensible fallbacks, not to turn an interface into a base class. Heavy logic in a default method is a smell that you actually wanted an abstract class or composition.

16. What happens when two interfaces define the same default method (the diamond problem), and how do you resolve it?

If a class implements two interfaces that both provide the **same default method**, the compiler cannot pick one, so it **forces you to override it** in the class. Inside that override you can delegate to a specific one with `InterfaceA.super.greet()`. So Java sidesteps the classic diamond problem by making the ambiguity a **compile error you must resolve explicitly**, rather than silently choosing. An abstract class can override a default method too, the same way any implementer can.



17. Why are interface static methods not inherited, and how do they differ from abstract-class static methods?

An **interface static method belongs to the interface itself** and is **not inherited** by implementing classes or sub-interfaces, so you call it as `MyInterface.method()`, never through an implementer. This keeps interface statics as namespaced helpers (like `Comparator.comparing(...)`). An **abstract class's static method is inherited** by subclasses (you can call it through the subclass, though it is hidden, not overridden). The difference comes from role: the interface static is a utility tied to the contract, the class static is an inherited class member.

18. What happens if a concrete class doesn't implement all interface methods?

It **will not compile**; a concrete class must implement every abstract method of the interfaces it declares. The escape hatch is to make the class **abstract itself**, which lets it implement an interface **without defining all the methods** and pass the rest down to its own subclasses. So either you implement everything, or you stay abstract and defer.

19. What is a functional interface, why `@FunctionalInterface`, and how does it differ from a normal interface?

A **functional interface has exactly one abstract method** (`Runnable`, `Comparator`, `Callable`), which is what makes it the target type for a **lambda**. The `@FunctionalInterface` annotation is optional but tells the compiler to **enforce the single-abstract-method rule**, so you get an error if someone later adds a second abstract method and breaks lambda use. A functional interface **can still have default and static methods**,

because those do not count against the "one abstract method" rule. The difference from a normal interface is only that constraint of a single abstract method.

20. Why do functional interfaces matter for lambdas, and what are the common ones?

They matter because a **lambda is an implementation of a functional interface's single method**: `x -> x > 0` is a `Predicate`. Without the single-method contract, the compiler would not know which method the lambda implements. The common ones in `java.util.function`:

Interface	Shape	Use
<code>Function<T,R></code>	T in, R out	Transform a value
<code>Consumer<T></code>	T in, nothing out	Do something with a value (side effect)
<code>Supplier<T></code>	nothing in, T out	Produce a value
<code>Predicate<T></code>	T in, boolean out	Test a condition

21. How do interfaces support loose coupling?

Because code can **depend on the interface, not the concrete class**. If a service takes a `PaymentGateway` interface rather than a specific `StripeGateway`, you can swap the implementation, inject a fake in tests, or add a new provider without touching the caller. The dependency points at the **contract**, so the concrete class becomes a detail you can change freely. This is the basis of dependency injection and most testable design.

22. If interfaces are more flexible, why do abstract classes still exist?

Because flexibility is not the only thing that matters; **shared implementation and state** are. An interface cannot hold instance fields or a constructor, so when several related classes genuinely share data and behaviour, an abstract class removes real duplication that interfaces (even with default methods) cannot. So they coexist: interface for the contract, abstract class for the shared guts, often together (contract as interface, a skeletal abstract base for the common parts).

23. You are designing a payment system. Would you use an abstract class or an interface, and why?

I would lead with an **interface**, something like `PaymentProvider` with `charge()`, `refund()`, `status()`. The reason is that the providers (Stripe, PayPal, a bank integration) are **unrelated implementations of the same capability**, and I want the rest of the system to depend on the contract so I can add or swap a provider without touching callers, and mock it in tests. If those providers turned out to share a real chunk of logic (the same request signing, retry, idempotency handling), I would add an **abstract base class** that implements the interface and holds that shared code, and let each provider extend it. So interface for the contract, abstract class underneath only if there is genuine shared implementation.

24. How would you refactor an inheritance-heavy codebase, and when do you replace one abstraction with the other?

The way I would approach it: find the places where **inheritance is being used just to share code** rather than to model a real "is-a", and pull that shared code into a **collaborator you compose in** instead (favour composition over inheritance). I would **replace an abstract class with an interface** when subclasses stopped sharing state

and the base had become an empty shell of abstract methods, and **replace an interface with an abstract class** the other way, when implementers had all copied the same default logic and it wanted a real shared base. I do it incrementally, behind tests, one hierarchy at a time, because deep inheritance trees break in surprising ways when you touch the base. *(Adapt to a real refactor you did.)*

25. Have you designed APIs using interfaces, and when was an abstract class the right call?

Yes, for anything meant to be **extended or swapped** I default to interfaces, because callers should depend on the contract, not my class, and interfaces mock cleanly in tests. A production case where an **abstract class was right**: a set of handlers that all shared the same setup, validation, and teardown, differing only in one step. That is a textbook **template-method** situation, so an abstract base owned the common flow and each handler filled in the one abstract step, which removed a lot of duplicated boilerplate that interfaces alone could not. *(Adapt to your own examples.)*

26. What guidelines would you set for your team on choosing between abstract classes, interfaces, composition, and inheritance, and what signals a wrong choice in review?

The guidelines I would give: **depend on interfaces** at boundaries; **prefer composition over inheritance** for sharing behaviour; use an **abstract class only for genuine shared state or a template-method flow**; and keep interfaces **small and focused** (interface segregation) rather than one god interface. In a **pull request** the signals that the wrong choice was made: a new abstract class with **no shared state and only abstract methods** (should be an interface), an interface with **a dozen unrelated methods** (should be split), **inheritance used purely to reuse a helper** (should be composition), and **business logic hidden in a default method** (should be a real class). When I see those, the abstraction is fighting the design rather than serving it.

27. Common mistakes.

- **Using an abstract class where an interface is enough**, locking implementers into a single-inheritance hierarchy for no gain.
- **Creating large "god interfaces"** that force implementers to define methods they do not need (violates interface segregation).
- **Adding mutable state to an abstract class** without a clear reason, which turns a base into a hidden source of bugs.
- **Overusing default methods for business logic**, when you actually wanted an abstract class or composition.
- **Using inheritance instead of composition** to share code, creating rigid hierarchies.
- **Assuming interfaces cannot evolve after Java 8**, when default methods exist precisely to let them.
- **Forgetting interface fields are always constants** (`public static final`), then expecting per-instance state.
- **Designing inheritance hierarchies that are too deep**, which are brittle and hard to reason about.